情報処理学会論文誌 Vol.0 No.0 1–2 (??? 1959)

CStamp-PWALによるWALボトルネックの緩和

著者名[1]

受付日 xxxx年0月xx日, 採録日 xxxx年0月xx日

概要：本稿では，トランザクション処理における write-ahead logging（WAL）のボトルネックを対象に，CStamp-PWAL の設計と評価を述べる．既存の ccbench-wal 実験基盤上で，WAL 処理，依存関係管理，およびコミット処理の内訳を測定し，スループット低下の要因を分析する．

Mitigating WAL Bottlenecks with CStamp-PWAL

Author Name[1]

Received: xx xx, xxxx, Accepted: xx xx, xxxx

Abstract: This paper studies write-ahead logging bottlenecks in transaction processing and evaluates the design of CStamp-PWAL on the ccbench-wal experimental framework. We break down WAL, dependency management, and commit processing costs to analyze the factors limiting throughput.

1. Introduction

1.1 Motivation

Modern main-memory transaction processing systems exploit many-core machines by removing centralized bottlenecks from concurrency control. Timestamp-based engines, including ERMIA-style protocols, already maintain a commit timestamp that determines the serialization order of committed transactions. As a result, the critical path of transaction execution is no longer determined only by validation or conflict management. When crash durability is required, the write-ahead logging (WAL) protocol can become the dominant scalability limit.

A conventional centralized WAL provides a simple and safe durability order, but it serializes log insertion and durable acknowledgment through a shared log stream. Parallel WAL (P-WAL) removes this obvious serialization point by partitioning log records across multiple WAL shards. However, once the shared WAL mutex is removed, the system exposes the next durability bottlenecks: storage synchronization, durable-point notification, and waiting for a global durable prefix. These costs are particularly important for IPSJ Transactions on Databases (TOD)-style database systems research, where the durability protocol must be evaluated as part of the end-to-end transaction processing design rather than as an isolated I/O optimization.

1.2 Problem

The key problem is that existing P-WAL designs often reintroduce a conservative global ordering mechanism at the durability layer. A transaction may have an ERMIA commit timestamp that already defines its serialization position, but the WAL layer separately allocates a global log sequence number (LSN) and acknowledges durability only when a global durable prefix reaches that LSN. This design is safe, but it couples the acknowledgment of unrelated transactions. A slow WAL shard can delay transactions on other shards even when they do not depend on any log record written by the slow shard.

[1] 慶應義塾大学
Keio University

The opposite design point is local-only acknowledgment: acknowledge a transaction as soon as its own WAL shard is durable. This removes the global-prefix wait and can improve throughput and latency, but it is unsafe in general. If transaction T reads data produced by transaction U, then acknowledging T before U's log record is durable can make recovery replay T without the state on which T depended. Therefore, the durability protocol must avoid both unnecessary global-prefix waiting and unsafe local-only acknowledgment.

1.3 Approach

This paper proposes CStamp-PWAL, a dependency-closed parallel WAL protocol for timestamp-based transaction processing. The first idea is to reuse the commit timestamp as a logical LSN. The WAL layer still maintains physical positions within each shard, but it does not need to allocate a separate global LSN for the recovery order. This removes a duplicated ordering mechanism between concurrency control and durability.

The second idea is dependency-closed durable acknowledgment. Instead of waiting for a global durable prefix, CStamp-PWAL acknowledges a transaction only after its own log record and the durable frontier of the transactions on which it depends have become durable. This condition preserves the dependency closure required for safe recovery while allowing independent WAL shards to progress without waiting for unrelated slow shards.

The third idea is to separate transaction execution from durable acknowledgment. Workers execute transactions, validate them, assign commit timestamps, and enqueue log records. Flushers batch and persist shard-local log records. Committers observe durable-frontier advances and acknowledge transactions whose dependency conditions are satisfied. This pipeline keeps worker threads from blocking directly on `fdatasync` or on conservative durable-prefix waits.

1.4 Organization

The rest of this paper is organized as follows. Section 2 reviews WAL, parallel WAL, and dependency requirements for crash recovery. Section 3 describes the design of CStamp-PWAL, including commit timestamps as logical LSNs, dependency-frontier construction, and the worker/flusher/committer pipeline. Section 4 presents the experimental methodology and evaluates the throughput, backlog, and tail-latency effects of CStamp-PWAL. Section 5 discusses related work, and Section 6 concludes the paper.

2. 背景

本節では，対象とするトランザクション処理基盤，WAL の役割，および既存方式で生じる性能上の課題を整理する．

3. 設計

CStamp-PWAL は，コミット時の依存関係と WAL 永続化の待ち合わせを分離し，高並列実行時の待機時間を削減することを狙う．本節では，ログレコードの扱い，依存関係の管理，およびコミット判定の流れを述べる．

4. 評価

評価では，YCSB，TPC-C，および WAL マイクロベンチマークを用いて，スループットとトランザクション内訳を測定する．実験結果は，`docs/` 配下の分析メモと `scripts/` 配下のプロット生成スクリプトから再現可能にする．

5. 関連研究

本節では，並行制御方式，WAL 最適化，グループコミット，および永続化待ち時間を隠蔽する既存手法との関係を整理する．

6. おわりに

本稿では，CStamp-PWAL の設計と評価を通じて，WAL ボトルネックを緩和するための実装上の要点を示した．今後は，より広いワークロードと永続化デバイス条件で評価を拡張する．

謝辞 本研究に関して有益な議論をいただいた関係者に感謝する．

参考文献